

PART A: Prerequisite

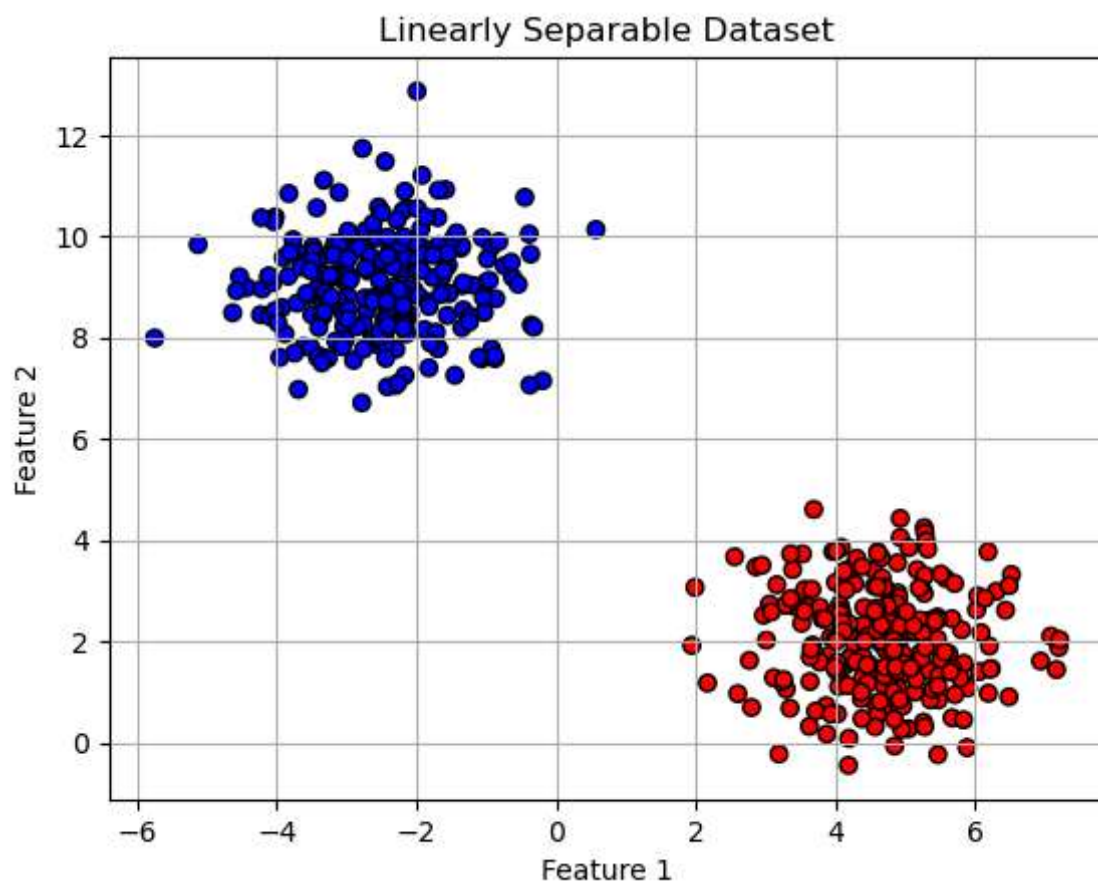
1.

```
In [10]: from sklearn.datasets import make_blobs, make_circles
import matplotlib.pyplot as plt
```

```
In [ ]: x, y = make_blobs(n_samples=500, centers=2, cluster_std=1.0, random_state=42)
```

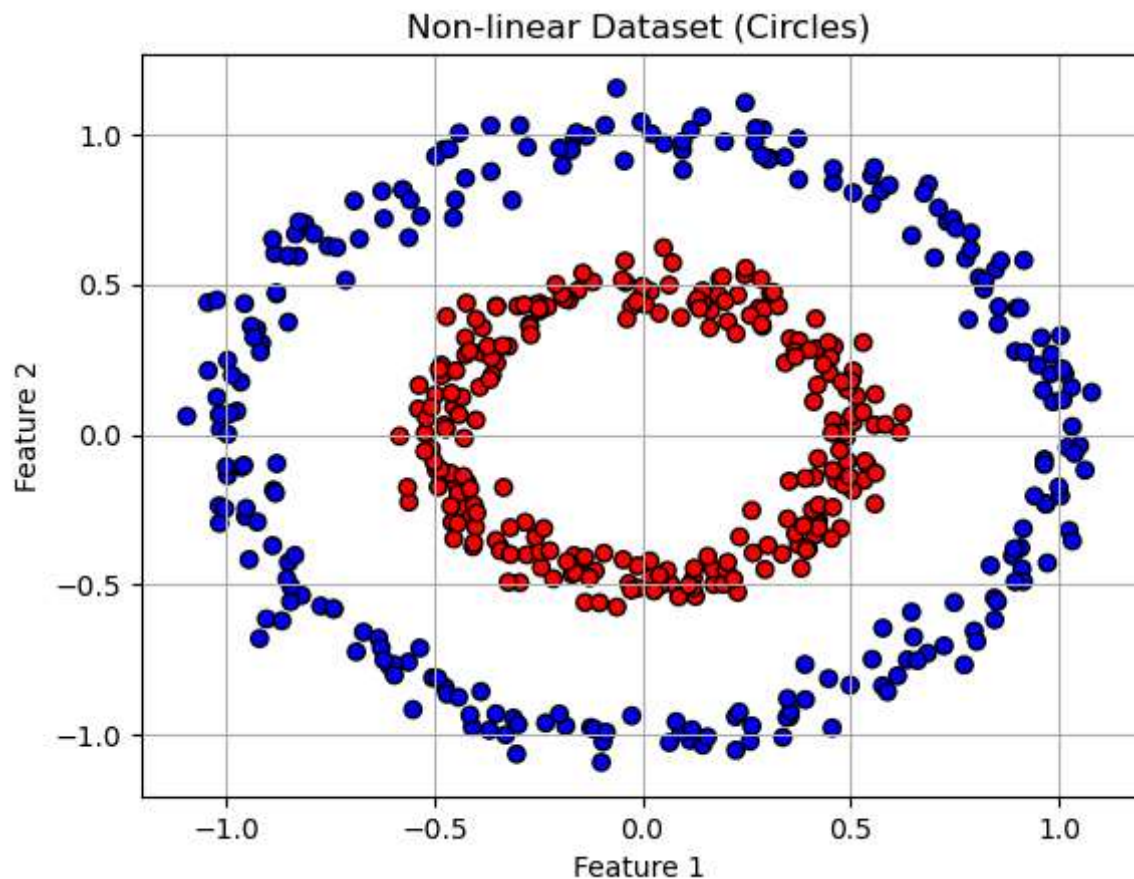
2.

```
In [70]: plt.scatter(x[:, 0], x[:, 1], c=y, cmap='bwr', edgecolor='k')
plt.title("Linearly Separable Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```



```
In [104...] X_circ, y_circ = make_circles(n_samples=500, factor=0.5, noise=0.05, random_state=42)
```

```
In [105...] plt.scatter(X_circ[:, 0], X_circ[:, 1], c=y_circ, cmap='bwr', edgecolor='k')
plt.title("Non-linear Dataset (Circles)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()
```



PART B: Linear SVM

1.

```
In [106... import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
In [107... X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
In [108... clf_linear = SVC(kernel='linear', C=0.1)
clf_linear.fit(X_train, y_train)
```

```
Out[108]: SVC
SVC(C=0.1, kernel='linear')
```

```
In [109... y_pred = clf_linear.predict(X_test)
```

2.

```
In [110... import numpy as np

# Create a mesh grid for plotting
xx, yy = np.meshgrid(
    np.linspace(x[:,0].min()-1, x[:,0].max()+1, 500),
    np.linspace(x[:,1].min()-1, x[:,1].max()+1, 500)
)
```

```

# Compute the decision function for each point in the grid
Z = clf_linear.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

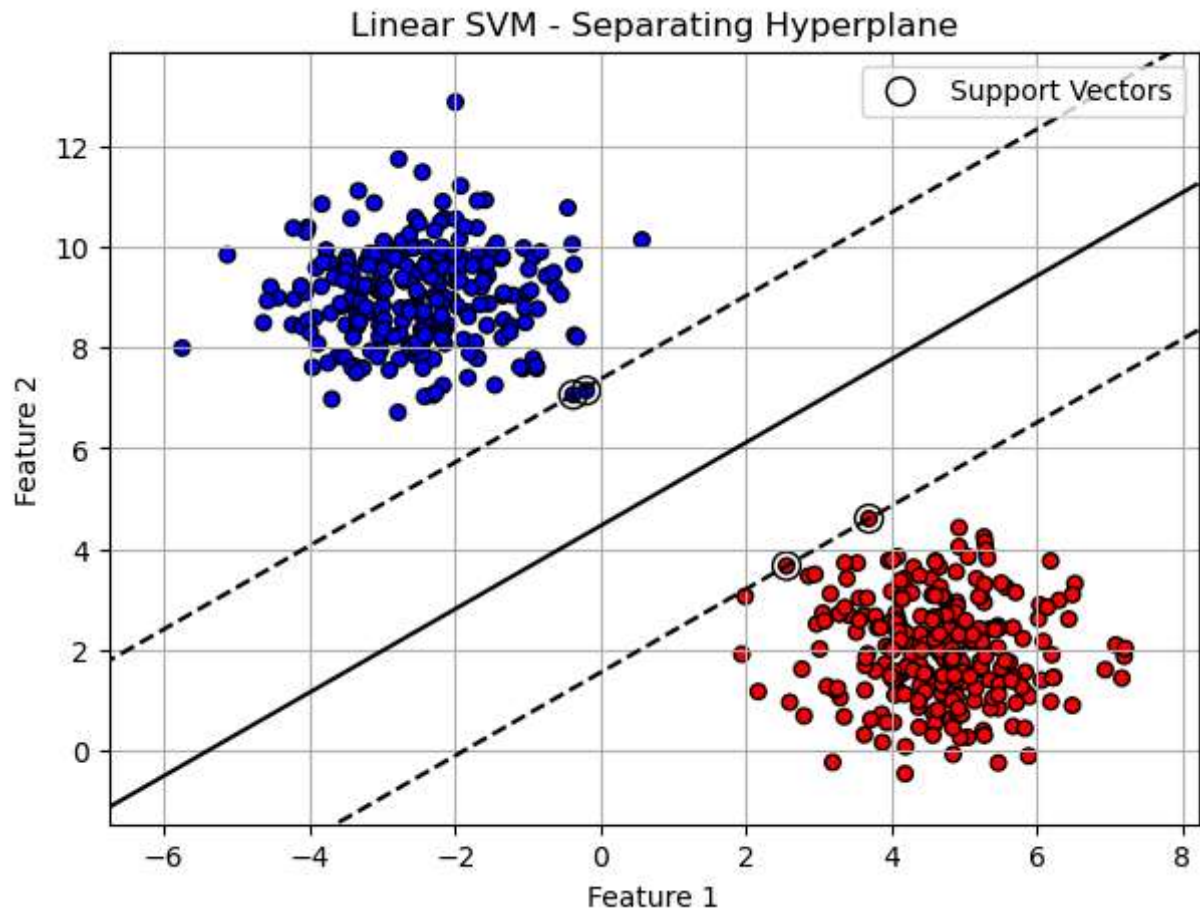
# Plot the points
plt.figure(figsize=(7,5))
plt.scatter(x[:,0], x[:,1], c=y, cmap='bwr', edgecolor='k', s=30)

# Plot the separating hyperplane
plt.contour(xx, yy, Z, levels=[0], colors='k') # decision boundary
plt.contour(xx, yy, Z, levels=[-1, 1], colors='k', linestyle='--') # margins

# Highlight support vectors
plt.scatter(clf_linear.support_vectors_[0], clf_linear.support_vectors_[1],
           s=100, facecolors='none', edgecolors='k', label='Support Vectors')

plt.title("Linear SVM - Separating Hyperplane")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True)
plt.show()

```



3.

```

In [111...] acc=accuracy_score(y_test,y_pred)
pre=precision_score(y_test,y_pred)
rec=recall_score(y_test,y_pred)
f1=f1_score(y_test,y_pred)

```

```

In [112...] print("Accuracy:", acc)
print("Precision:", pre)

```

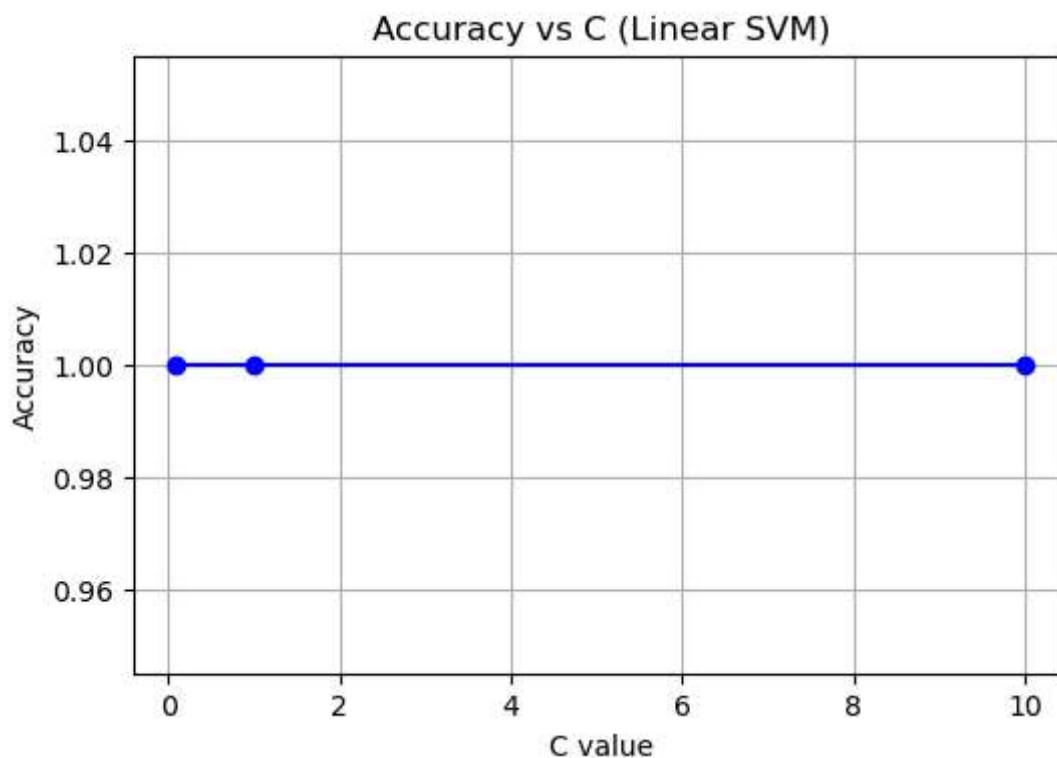
```
print("Recall:", rec)
print("F1-score:", f1)
```

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1-score: 1.0

4.

In [113...

```
C_values = [0.1, 1, 10]
accuracies = []
for C in C_values:
    clf = SVC(kernel='linear', C=C)
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred))
plt.figure(figsize=(6,4))
plt.plot(C_values, accuracies, marker='o', color='blue')
plt.title("Accuracy vs C (Linear SVM)")
plt.xlabel("C value")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()
```



PART C: Non-linear SVM

1.

In [114...

```
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_circ, y_circ, test_size=0.3, random_state=42
)
```

In [115...

```
kernels = ['poly', 'rbf', 'sigmoid']
results = {}
```

```

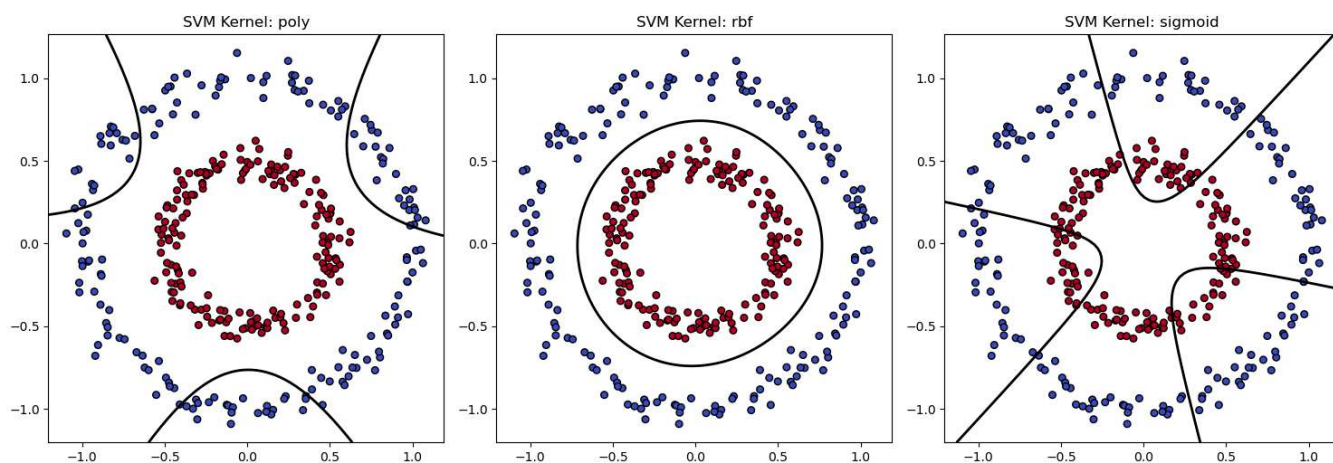
In [127... plt.figure(figsize=(15,6)) # taller figure
for i, k in enumerate(kernels, 1):
    clf = SVC(kernel=k, C=1)
    clf.fit(X_train_c, y_train_c)
    y_pred_c = clf.predict(X_test_c)
    acc = accuracy_score(y_test_c, y_pred_c)
    results[k] = acc

    ax = plt.subplot(1, 3, i)
    ax.set_aspect('equal', adjustable='box') # make plot square
    plt.title(f"SVM Kernel: {k}")
    plt.scatter(X_train_c[:,0], X_train_c[:,1], c=y_train_c,
                cmap='coolwarm', edgecolors='k', s=30)

    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
    xx = np.linspace(xlim[0], xlim[1], 100)
    yy = np.linspace(ylim[0], ylim[1], 100)
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = clf.decision_function(xy).reshape(XX.shape)
    plt.contour(XX, YY, Z, levels=[0], linewidths=2, colors='k')

plt.tight_layout()
plt.show()

```



```

In [ ]: print("\nKernel Comparison:")
for k, acc in results.items():
    print(f"Kernel = {k:8s} → Accuracy = {acc:.3f}")

```

```

Kernel Comparison:
Kernel = poly      → Accuracy = 0.680
Kernel = rbf      → Accuracy = 1.000
Kernel = sigmoid  → Accuracy = 0.493

```

2.

```

In [128... from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

```

```

In [129... clf_cv = SVC(kernel='rbf', C=1)
scores = cross_val_score(clf_cv, X_circ, y_circ, cv=5, scoring='accuracy')

```

```

In [130... print("\n5-Fold Cross Validation Scores:", scores)
print("Mean Accuracy:", scores.mean())
print("Standard Deviation:", scores.std())

```

5-Fold Cross Validation Scores: [1. 1. 1. 1. 1.]

Mean Accuracy: 1.0

Standard Deviation: 0.0

```
In [131... clf_log = LogisticRegression()  
clf_log.fit(X_circ, y_circ) # Train on full dataset  
y_pred_log = clf_log.predict(X_circ)  
acc_log = accuracy_score(y_circ, y_pred_log)
```

```
In [132... print("\nAccuracy Comparison:")  
print(f"SVM (RBF): {scores.mean():.3f}") # mean CV accuracy  
print(f"Logistic Regression: {acc_log:.3f}")
```

Accuracy Comparison:

SVM (RBF): 1.000

Logistic Regression: 0.498